

Pumping-Lemma: einfaches Beispiel

Methodik / Definition.

Pumping-Lemma zum Nachweis von Nicht-Regularität:

1. Annahme: Die Sprache ist regulär. Sei p die Pumping-Länge.
2. Wähle ein Wort s aus der Sprache mit $|s| \geq p$.
3. Betrachte eine beliebige Zerlegung $s = xyz$ mit $|y| > 0$ und $|xy| \leq p$.
4. Zeige für ein geeignetes i meistens $i = 0$ oder $i = 2$, dass xy^iz nicht mehr in der Sprache liegt.

Das ist ein Widerspruch. Also ist die Sprache nicht regulär.

Wir betrachten die Sprache

$$L = \{w \in \{a, b\}^* \mid |w|_a > |w|_b\}.$$

Dabei bezeichnet $|w|_a$ die Anzahl der a 's in w und $|w|_b$ die Anzahl der b 's in w .

Warum reichen mehr Zustände nicht?

Ein Automat für L müsste sich merken, ob bisher mehr a 's als b 's gelesen wurden. Dafür ist die Differenz

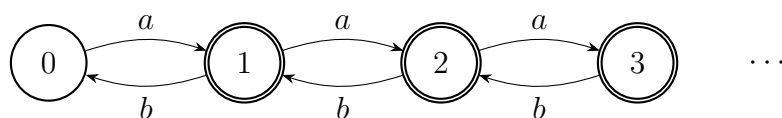
$$|w|_a - |w|_b$$

entscheidend. Schon bei den Wörtern

$$\varepsilon, a, aa, aaa, aaaa, \dots$$

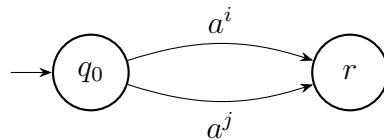
entstehen unendlich viele mögliche Differenzen:

$$0, 1, 2, 3, 4, \dots$$



Die Zustände $1, 2, 3, \dots$ wären akzeptierend, weil dort mehr a 's als b 's gelesen wurden. Der Zustand 0 wäre nicht akzeptierend, weil dort gleich viele a 's und b 's gelesen wurden.

Ein endlicher Automat kann diese unendlich vielen Differenzen nicht alle getrennt speichern. Also muss er irgendwann zwei verschiedene Differenzen im selben Zustand zusammenfassen:



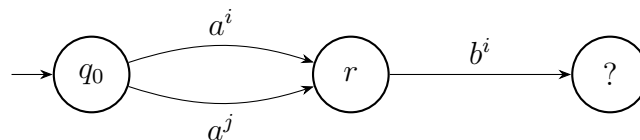
$i < j$
aber gleicher Zustand

Ab diesem Zustand kann der Automat nicht mehr wissen, ob er eigentlich Differenz i oder Differenz j gespeichert hatte. Aber diese zwei Situationen brauchen verschiedene Antworten. Hänge nämlich b^i an:

$$a^i b^i \notin L \quad \text{aber} \quad a^j b^i \in L,$$

denn

$$i - i = 0 \quad \text{aber} \quad j - i > 0.$$

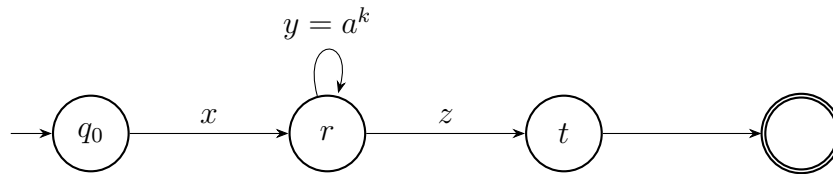


müsste gleichzeitig
ablehnen und akzeptieren

Das ist unmöglich. Mehr Zustände verschieben das Problem nur weiter nach hinten. Für alle möglichen Differenzen bräuchte man unendlich viele Zustände.

Warum darf man pumpen?

Das Pumping-Lemma kommt aus einer Eigenschaft endlicher Automaten: Ein endlicher Automat hat nur endlich viele Zustände. Wenn er ein langes Wort liest, muss er irgendwann einen Zustand wiederholen. Dann ist der Teil des Wortes zwischen den beiden Besuchen eine Schleife.



Wenn der Automat nach x und nach xy im selben Zustand r ist, kann er nicht unterscheiden, ob die Schleife y einmal, zweimal oder gar nicht gelesen wurde. Deshalb muss er alle Wörter

$$xz, \quad xyz, \quad xy^2z, \quad xy^3z, \dots$$

gleich behandeln.

Genau hier entsteht der Widerspruch: Wenn ein Automat xyz akzeptiert, dann müsste er wegen der Schleife auch xz akzeptieren. Aber xz liegt nicht mehr in der Sprache. Der Automat würde also ein falsches Wort akzeptieren. Darum kann es keinen endlichen Automaten für diese Sprache geben.

Beweis.

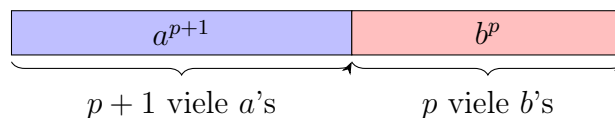
1. Annahme: L ist regulär. Sei p die Pumping-Länge.
2. Wähle

$$s = a^{p+1}b^p.$$

Dann gilt

$$|s|_a = p + 1 > p = |s|_b,$$

also ist $s \in L$. Außerdem gilt $|s| = 2p + 1 \geq p$.

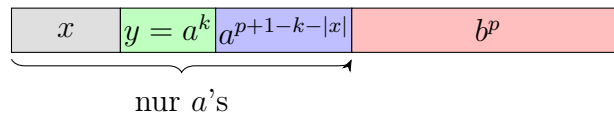


3. Sei $s = xyz$ eine Zerlegung gemäß Pumping-Lemma, also

$$|y| > 0 \quad \text{und} \quad |xy| \leq p.$$

Weil die ersten p Zeichen von s alle a sind, liegt y vollständig im ersten a -Block. Also gilt

$$y = a^k \quad \text{für ein } k \geq 1.$$



4. Wir pumpen mit $i = 0$, also entfernen wir y :

$$xy^0z = xz = a^{p+1-k}b^p.$$

5. Jetzt gilt

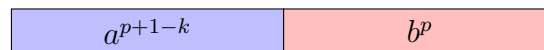
$$|xz|_a = p + 1 - k \quad \text{und} \quad |xz|_b = p.$$

Wegen $k \geq 1$ folgt

$$p + 1 - k \leq p.$$

Also hat xz nicht mehr a 's als b 's:

$$xz \notin L.$$



$$p + 1 - k \leq p \text{ viele } a\text{'s} \quad p \text{ viele } b\text{'s}$$

Das ist ein Widerspruch zum Pumping-Lemma.

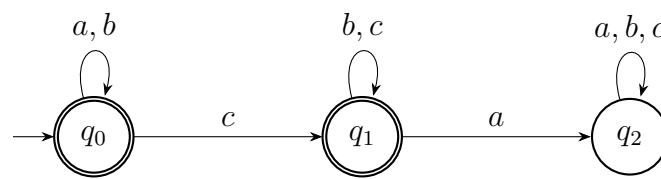
Folglich ist L nicht regulär.

Warum der Beweis bei einem regulären Automaten nicht funktioniert

Jetzt betrachten wir zum Vergleich eine Sprache aus Aufgabenblatt 1:

$$L_1 = \{w \in \{a, b, c\}^* \mid \text{kein } c \text{ steht vor einem } a\}.$$

Diese Sprache ist regulär. Ein passender Automat merkt sich nur, ob schon ein c gelesen wurde:



q_0 bedeutet: Es wurde noch kein c gelesen. q_1 bedeutet: Es wurde schon ein c gelesen, aber danach noch kein a . q_2 ist die Falle: Es kam ein a nach einem c .

Versuchen wir trotzdem, einen Pumping-Lemma-Beweis wie oben zu bauen. Sei p die Pumping-Länge und wähle

$$s = a^p c^p.$$

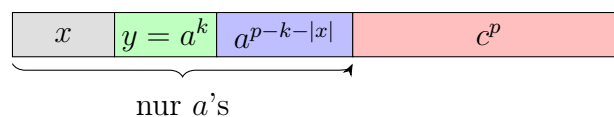
Dann gilt $s \in L_1$, denn alle a 's stehen vor allen c 's.

Sei nun $s = xyz$ eine Zerlegung mit

$$|y| > 0 \quad \text{und} \quad |xy| \leq p.$$

Weil die ersten p Zeichen von s alle a sind, liegt y vollständig im ersten a -Block. Also gilt

$$y = a^k \quad \text{für ein } k \geq 1.$$



Pumpen mit $i = 0$ entfernt nur einige a 's:

$$xy^0z = xz = a^{p-k}c^p.$$

Dieses Wort liegt aber immer noch in L_1 , weil weiterhin kein c vor einem a steht.

Auch Pumpen mit $i = 2$ hilft nicht:

$$xy^2z = a^{p+k}c^p.$$

Das Wort liegt ebenfalls noch in L_1 .

Der Versuch scheitert also genau an der Stelle, an der beim nicht-regulären Beispiel der Widerspruch entsteht. Das Pumpen verändert nur die Anzahl der a 's am Anfang, aber nicht die Form

$$\{a, b\}^* \{b, c\}^*.$$

Deshalb bekommt man kein Wort außerhalb der Sprache.

Wichtig: Das Pumping-Lemma ist ein Werkzeug, um Nicht-Regularität zu zeigen. Bei regulären Sprachen darf ein solcher Widerspruch nicht entstehen. Hier funktioniert der Pumping-Lemma-Beweis also nicht, weil L_1 tatsächlich regulär ist.